

MANUAL DE GIT PARA EL EQUIPO

Proyecto Java — POO con JDBC y Swing

Guía para trabajar en equipo con GitHub

 main	 develop	 feature/...
Código estable final. Nadie toca esta rama directamente.	Rama de integración del equipo. Base para trabajar.	Tu rama personal. Una por cada tarea que hagas.

1. Clonar el repositorio (solo una vez)

Haces esto una sola vez en tu computadora. Te descarga todo el proyecto de GitHub.

```
git clone https://github.com/tuusuario/nombre-del-repo.git
```

```
# Luego entra a la carpeta del proyecto:
```

```
cd nombre-del-repo
```



Después del clone, ya no vuelves a correr ese comando. Para actualizar tu código usa git pull.

2. Antes de empezar a trabajar (siempre)

Cada vez que vayas a trabajar, primero baja los cambios que hicieron tus compañeros.

1

Ir a la rama develop

```
git checkout develop
```

2 Bajar los últimos cambios

```
git pull
```

3 Crear tu rama para la tarea

```
# Cambia el nombre según tu tarea, por ejemplo:
```

```
git checkout -b feature/ventana-login
```

```
git checkout -b feature/clase-usuario
```

```
git checkout -b feature/conexion-db
```



Siempre parte de develop actualizado antes de crear tu rama. Así evitas conflictos.

3. Mientras trabajas

Mientras programas, guarda tu avance en Git con commits. Hazlo seguido, no esperes a terminar todo.

1 Ver qué archivos cambiaste

```
git status
```

2 Agregar tus cambios

```
# Agregar todos los archivos modificados:
```

```
git add .
```

```
# O agregar un archivo específico:

git add src/com/equipo/vista/VentanaLogin.java
```

3 Guardar el commit con un mensaje claro

```
git commit -m "agrego formulario de login con validación"

git commit -m "creo clase Usuario con sus atributos"

git commit -m "conecto a base de datos MySQL"
```

4 Subir tu rama a GitHub

```
# La primera vez que subes tu rama:

git push origin feature/ventana-login

# Las siguientes veces (ya está creada):

git push
```

✗ Mensajes malos	☑ Mensajes buenos
"cambios"	"agrego ventana de registro"
"arregle cosas"	"corrijo error al conectar a MySQL"
"avance"	"completo CRUD de productos"
"fix"	"valido campos vacíos en formulario"

4. Cuando terminas tu tarea

Una vez que tu funcionalidad está lista y subida, abres un Pull Request en GitHub para que tu líder la revise y la una a develop.

1 Asegúrate de que todo está subido

```
git add .  
  
git commit -m "finalizo ventana de login"  
  
git push
```

2 Abrir un Pull Request en GitHub

1. Ve al repositorio en GitHub.com
2. Aparecerá un botón amarillo "Compare & pull request" — haz clic
3. Asegúrate que va de tu rama → develop (no a main)
4. Escribe un título claro y haz clic en "Create pull request"
5. Espera a que el líder lo apruebe y lo una a develop



NUNCA hagas el Pull Request hacia main. Siempre hacia develop.

5. Para empezar una nueva tarea

Cuando termines una tarea y hagas el Pull Request, empieza la siguiente desde cero con una rama nueva.

```
# Regresa a develop y actualiza  
  
git checkout develop  
  
git pull  
  
  
# Crea una nueva rama para tu siguiente tarea  
  
git checkout -b feature/nombre-nueva-tarea
```



Una rama por tarea. No reutilices ramas antiguas.

6. Resumen del flujo diario

#	Comando	Para qué sirve
1	git checkout develop	Ir a la rama de integración
2	git pull	Bajar cambios de tus compañeros
3	git checkout -b feature/tarea	Crear tu rama personal
4	git add .	Preparar tus archivos para guardar
5	git commit -m "mensaje"	Guardar un punto de control
6	git push	Subir tu trabajo a GitHub
7	Pull Request en GitHub	Pedir que tu código se una a develop

7. Reglas de oro del equipo

✗	NUNCA hagas push directo a main ni a develop
✗	NUNCA trabajes en la rama de otro sin avisarle
✓	SIEMPRE haz git pull antes de empezar a trabajar
✓	SIEMPRE crea una rama nueva por cada tarea
✓	SIEMPRE escribe mensajes de commit claros y descriptivos
✓	SIEMPRE espera aprobación antes de fusionar tu PR

¿Dudas? Pregúntale al líder del proyecto antes de hacer algo que no estés seguro.